



Path planning and collision avoidance based on the RRT*FN framework for a robotic manipulator in various scenarios

Jianyou Qi¹ · Qingni Yuan^{1,2} · Chen Wang¹ · Xiaoying Du¹ · Feilong Du^{1,2} · Ao Ren¹

Received: 19 March 2023 / Accepted: 27 May 2023 / Published online: 6 July 2023
© The Author(s) 2023

Abstract

In this article, we present a new path planning algorithm based on the rapidly exploring random tree-fixed node (RRT*FN) algorithm for manipulators. It addresses the problem that RRT*FN complex environment processing is not fast enough to meet real-time requirements, and RRT*FN is basically impossible to search for an effective manipulator path in a narrow-channel environment. In the new path planning algorithm, a heuristic sampling method is adopted and the leaf nodes outside the ellipsoids are preferentially deleted to address the node removal problem, resulting in better search paths and faster search speeds. When the nodes are expanded, new nodes are generated due to goal gravity and random point gravity, with the weight values dynamically adjusted via dichotomy, and the new nodes are expanded to the goal point twice to more rapidly obtain a tree extension direction that is closer to the goal point. For boundary points, the issues of narrow channels and stepped obstacles can be effectively solved by extending the local environment sampling boundaries. To optimize the paths, the redundant intermediate nodes are simplified based on the triangle inequality. Simulation analyses show that the proposed planner can adapt to a variety of scenarios in real time while satisfying optimal path conditions.

Keywords Boundary extension · Collision avoidance · Manipulator · Path planning · Rapidly exploring random tree-fixed node (RRT*FN) · Various scenarios

Introduction

Manipulators are playing an increasingly important role in many fields, such as the handling and sorting of goods in

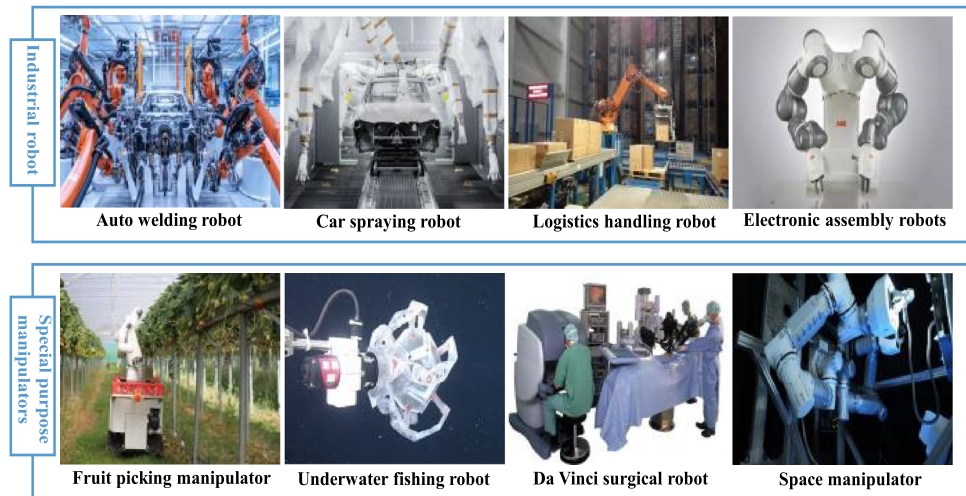
industry, as shown in Fig. 1. However, with the expansion of this field, the environment surrounding home or industrial logistics manipulators is no longer a fixed and simple environment such as that of a traditional industrial mechanical arm [1] but is rather a more complex and diverse environment. Therefore, the robotic arm path planning algorithm has more stringent requirements to ensure that it can adapt to various complex environments. For this, researchers have applied neural network intelligence algorithms, adaptive control and algorithms related to rapidly exploring random trees (RRTs) to control manipulator obstacle avoidance and motion planning. Khan et al. designed a metaheuristic-based control framework called the beetle antennae olfactory recurrent neural network, and achieved simultaneous tracking control and avoidance of arbitrarily shaped obstacles to a certain extent [2]. Xu et al. investigated the formulation of the motion planning problem as a Quadratic Planning (QP) problem and used Recurrent Neural Networks (RNNs) to achieve real-time manipulator planning in complex workspaces, which provides a new idea for real-time manipulator motion planning for simultaneous obstacle avoidance and target tracking [3]. However, neural networks

✉ Qingni Yuan
qnyuan@gzu.edu.cn
Jianyou Qi
scienceqjy@163.com
Chen Wang
944068633@qq.com
Xiaoying Du
1831661907@qq.com
Feilong Du
du_fei_long@163.com
Ao Ren
3330164043@qq.com

¹ Key Laboratory of Modern Manufacturing Technology, Ministry of Education, West Campus, Guizhou University, Huaxi District, Guiyang City, Guizhou Province, China

² School of Mechanical Engineering, Guizhou University, Guiyang 550000, China

Fig. 1 The field in which the manipulator is used



are poor in interpretation, and RNNs require a large amount of sample data to train suitable network parameters. The process is extremely time consuming, and the performance of RNNs in narrow-passage obstacle avoidance has not been fully demonstrated. Furthermore, in the field of adaptive control for motion planning of manipulators, to address various environment uncertainties, attempts to achieve more accurate control and iterative learning-based approaches have been proposed by researchers to complete adaptive optimal control [4, 5]. Some scholars try to plan the path only by building a manipulator motion model and some studies targeted models to address the disturbances, modeling errors and various uncertainties of the model in real systems [6]. However, the flexibility of the above algorithms is limited, and the calculation efficiency is low.

Currently, the sampling-based rapidly RRT algorithm planning method [7–10], which is more suitable for higher dimensional space motion planning, is usually used for manipulator path planning.

To realize real-time and optimal path planning, various researchers have creatively improved the RRT algorithm. For example, the RRT* algorithm [11] introduces a cost function into the RRT algorithm to produce an asymptotically optimal planned path, which increases the planning time. Zhou et al. developed a novel strategy for searching the nearest neighbors named the generalized distance method that reduces the search time [12]. Adiyatov et al. proposed a rapidly exploring random tree-fixed node (RRT*FN) algorithm that limits the maximum number of nodes, with the same sampling, node expansion and parent node selection methods of the RRT* algorithm. Although this method prevents infinite tree growth and saves memory, the planning speed is not greatly improved, and the convergence accuracy is low [13]. Tian et al. attempted to determine the parent node before generating new points to eliminate the complex process of traversing a random tree to search for the parent node.

However, this method increases the likelihood of becoming trapped in a local minimum [14]. Shen et al. proposed a novel manipulability-based optimal rapidly exploring random tree (RRT*) path planning strategy and achieved the shortest path to the target location. However, it does not involve obstacle collisions and is only suitable for unobstructed environments [15].

In view of the increasing robotic arm environment complexity, researchers have made further improvements to the above algorithms. Liu et al. proposed selecting the goal point with a certain probability to produce new nodes, and other new nodes were generated as a result of the combined action of the goal gravity and random point gravity. As a result, this is conducive to exploring a better path and ensures that the goal point can be reached quickly. However, the expansion step is fixed in both directions, increasing the difficulty of adapting to different environments [16]. Yi et al. improved the P_RRT* method, proposing two methods for selecting the nearest neighbor nodes to increase the convergence speed [17]. Due to the excessive size of the search space, Li et al. adopted a regression filtering mechanism and proposed a random point boundary expansion mechanism, effectively saving time. However, the extension nodes that attempt to pass through the narrow channel will be excluded from the tree, and thus, a collision-free path cannot be planned [18]. Yang et al. selected multiple random points as candidates and chose the one closest to the goal node as the sampling point, slightly increasing the convergence speed to some extent. But this process increases the time cost of invalid comparisons with sampling points in the same region; thus, it has little effect on the overall convergence speed [19]. Fang et al. proposed an ellipsoid subset sampling method, which not only accelerates the convergence but also ensures a better search path. When no feasible solution exists in the ellipsoid, the planning task fails [20]. Liu et al. proposed a time-optimal rapidly exploring random tree (TO-RRT)

algorithm for a picking robot, and the planning speed was increased. However, the TO-RRT algorithm produced larger steps near obstacles, which led to a longer path length, and multi-obstacle environments are not suitable [21]. Jiang et al. proposed the RRT_Connect approach in collaboration with an artificial potential field method, and they attempted to use environment and nodes information to reduce the excessive exploration. Although it is effective to address a certain number of obstacles, the types of obstacles are similar in that the distance between obstacles is still wide enough, and it has only been demonstrated in three-axis manipulators in their study [22].

Environment complexity inevitably results in issues with narrow-channel path planning, which is still in early development. Kuffner et al. used bidirectional trees to search the state space and proposed the standard RRT-Connect algorithm, which greatly improves the search efficiency and provides a solution for path planning with narrow passages on one side. However, it does not provide an optimal solution and is nearly identical to the RRT algorithm in an environment with narrow channels on both sides [23]. Wu et al. proposed a random turn expansion strategy that randomly generates an expansion direction after the expansion based on the direction of random sampling point failures. It can sometimes address the narrow-channel path planning problem, but is highly random [24]. Li et al. developed an environmental evaluation method to guide the selection of the correct expansion direction. This method greatly improves path planning in narrow-channel environments. However, it fails to completely address the problems caused by the environment type [25]. Zhang et al. proposed a motion planning algorithm for a hyper-redundant manipulator in narrow spaces, but it only applies to hyper-redundant manipulators, and its premise requires a known path [26].

In response to the above problems, an improved path planning algorithm that can be adapted to different scenarios is proposed in this paper. This algorithm considers both general complex environments and narrow-passage environments. The main contributions of this work are as follows:

1. Aiming at the time-consuming problem of path searching in complex scenes, a method for greedy expansion based on dichotomy and double expansion and a method for multi-region heuristic sampling are introduced to achieve the effect of real-time path planning.
2. Under the premise of requiring real-time performance, when no path search results exist in extreme situations such as narrow passages, a strategy for boundary extension based on local environment sampling is proposed. Leaf nodes outside the ellipsoid defined by the starting and goal points as the two focal points are prioritized for removal.
3. To obtain a better path, the triangular inequality principle is used to reselect the parent node for each node in

the obtained path, simplify the intermediate redundant nodes, and optimize the final path.

The remainder of this paper is organized as follows. The “Preliminaries” section includes two subsections, the “Kinematics model and solution for the manipulator” subsection, which studies a solution strategy of bypassing singular values and establishes a manipulator kinematics model, and the “Problem formulation and practical use of path planning”, which describes the manipulator path planning problem and explains the practical use of the path planning research in this paper. The “Manipulator path planning based on the RRT*FN” section focuses on the path planning strategy of the manipulator based on the RRT*FN framework proposed in this paper, including four main contributions. The simulation results of the proposed algorithm in two-dimensional and three-dimensional spaces, as well as experiments using the six degrees-of-freedom (DOF) of the manipulator in the robotic operating system (ROS), are described in detail in the “Experimental results” section. Finally, concluding remarks are presented in the “Conclusion” section.

Preliminaries

Kinematics model and solution for the manipulator

Because directly mapping obstacles to the joint space is difficult, some researchers have performed obstacle avoidance operations in a Cartesian coordinate system, with the remaining operations performed in the joint space, i.e., the C-space [27], inevitably increasing the space switching cost. Therefore, in this study, path planning is carried out entirely in a Cartesian coordinate space.

A six-DOF series manipulator is used as a prototype for the algorithm correlation analysis, and its three-dimensional model is shown in Fig. 2a. Its kinematics model is established using the Denavit–Hartenberg (D–H) parameter method [28] and shown in Fig. 2b. The joint variables and connecting rod parameters are shown in Table 1.

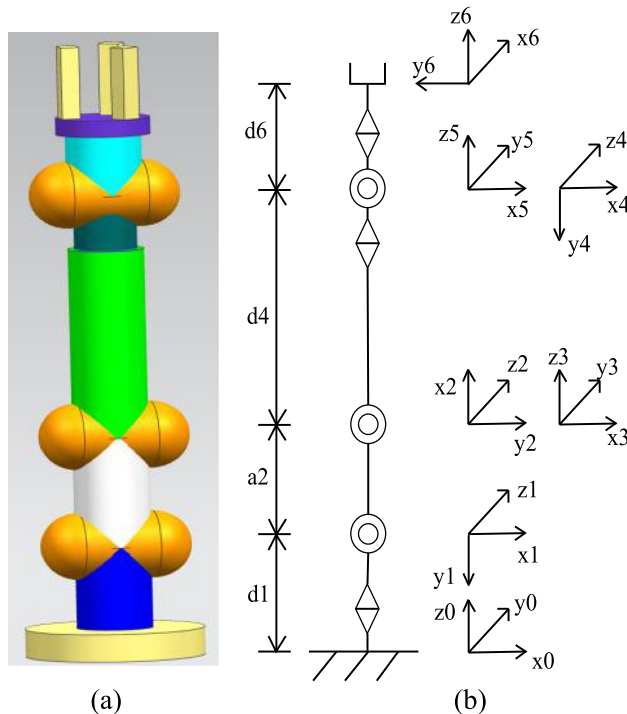
Since the manipulator wrist joint axes intersect at one point and satisfy the Pieper criterion, a closed solution exists. Eight sets of inverse solutions can be obtained. However, in practical applications, the optimal solution is generally selected based on the minimum change in the joint angle, namely, the θ_i solution satisfying $\min \sum_{i=1}^6 |\theta_{\text{goal},i} - \theta_{\text{init},i}|$.

For other types of manipulators, during the planning process, the inverse kinematics problem may not be solvable with analytical (algebraic or geometric) or numerical methods. In this case, short-distance local reinforcement learning [29] can be used to address the inverse kinematics of the manipulator. A reinforcement learning method is temporarily used to plan the path during the local planning process so

Table 1 D–H parameters of the six-DOF manipulator

i	a_i/mm	$\alpha_i/(^{\circ})$	d_i/mm	$\theta_i/(^{\circ})$	$\beta_i/(^{\circ})$
1	0	-90	140	θ_1	± 180
2	140	0	0	θ_2	± 135
3	0	90	0	θ_3	$-150 \sim +180$
4	0	-90	280	θ_4	± 170
5	0	90	0	θ_5	± 119
6	0	0	120	θ_6	± 360

The parameter β_i is the limit of joint i

**Fig. 2** Six-DOF manipulator D–H coordinate system. **a** Three-dimensional model. **b** Mechanical structure diagram of the manipulator and D–H coordinate system

the robotic arm can either reach the singular value position according to the previous position of the singular value point obtained using the proposed algorithm to determine the corresponding angle of each joint at the singular value position or advance directly to the singular value point and enter the next algorithm starting stage, which is not described here.

Problem formulation and practical use of path planning

Before the manipulator path can be planned, the path planning problem and the symbols used during planning must be defined. Let $S \subset \mathbf{R}^3$ be the state space. The space colliding with obstacles is defined as $S_{obs} \subset S$, and the space not colliding with obstacles is defined as $S_{free} = S \setminus S_{obs}$, that is, the

free space. The path planning starting point, that is, the initial position state, is $s_{init} \in S_{free}$, and the end point, that is, the target position state, is $s_{goal} \in S_{free}$. Let $\sigma : [0, 1] \rightarrow \mathbf{R}^3$ be a continuous function representing points on a path between the starting position and the target area, with $\sigma(0) = s_{init}$ and $\sigma(1) = s_{goal}$. If all α constituting a path satisfy $\forall \alpha \in [0, 1]$, $\sigma(\alpha) \in S_{free}$, the path is considered to be collision-free; furthermore, if α can take values of 0 or 1 with $\sigma(0) = s_{init}$ and $\sigma(1) = s_{goal}$, then the collision-free path is a feasible path planning solution.

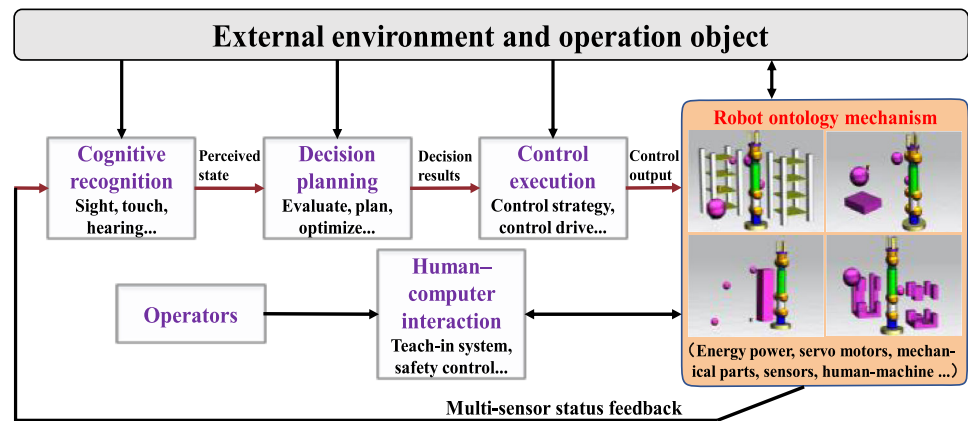
To determine the optimal path, the path cost is defined as a monotonically bounded nonnegative function $c: S_{free} \rightarrow \mathbf{R}$ for all collision-free paths, where $\mathbf{R}$ represents the set of nonnegative real numbers. The optimal path planning goal is to determine the path with the lowest path cost until the algorithm determines the optimal path, corresponding to the lowest cost. The best feasible path σ_{best} between the initial position and target position is defined as:

$$\sigma_{best} = \arg \min_{\sigma \in \Sigma} \{c(\sigma) | \sigma(0) = s_{init}, \sigma(1) = s_{goal}, \forall \alpha \in [0, 1], \sigma(\alpha) \in S_{free}\}, \quad (1)$$

where Σ represents the set of all collision-free paths that have been searched.

If there is a state space subset S_f satisfying: $S_f = \{s \in S | f(s) < c_{pre_best}\}$ and $S_f \subseteq S$, where c_{pre_best} is the cost of the current best path, then a better solution than the existing solutions can be found in S_f . In addition, the state of S_f is less than that of the entire state space S ; thus, if the search area is limited to S_f during planning, the algorithm can determine a near-optimal solution more quickly.

As shown in Fig. 3, the focus of this paper is to solve the decision planning part for home service robots or logistics handling and sorting robots in the face of a variety of nonstructural complex environments. To the planning should effectively avoid obstacles to quickly plan the path with the lower cost path, control the manipulator movement to the specified position, and prepare for subsequent grabbing and other operations. The algorithm designed here needs to be

Fig. 3 Intelligent robotic arm system components

adaptable to a variety of environments, especially for special scenarios such as narrow passages, and can also plan collision-free paths in a short time. The length of time is relative, and this study takes real time as a reference indicator for a very short time, which can fully meet the average shortest time spent by the robotic arm in the operating environment to ensure normal operation completion. And the short time will also prepare for a subsequent study of having time to react to a dynamic environment and avoid dynamic obstacles.

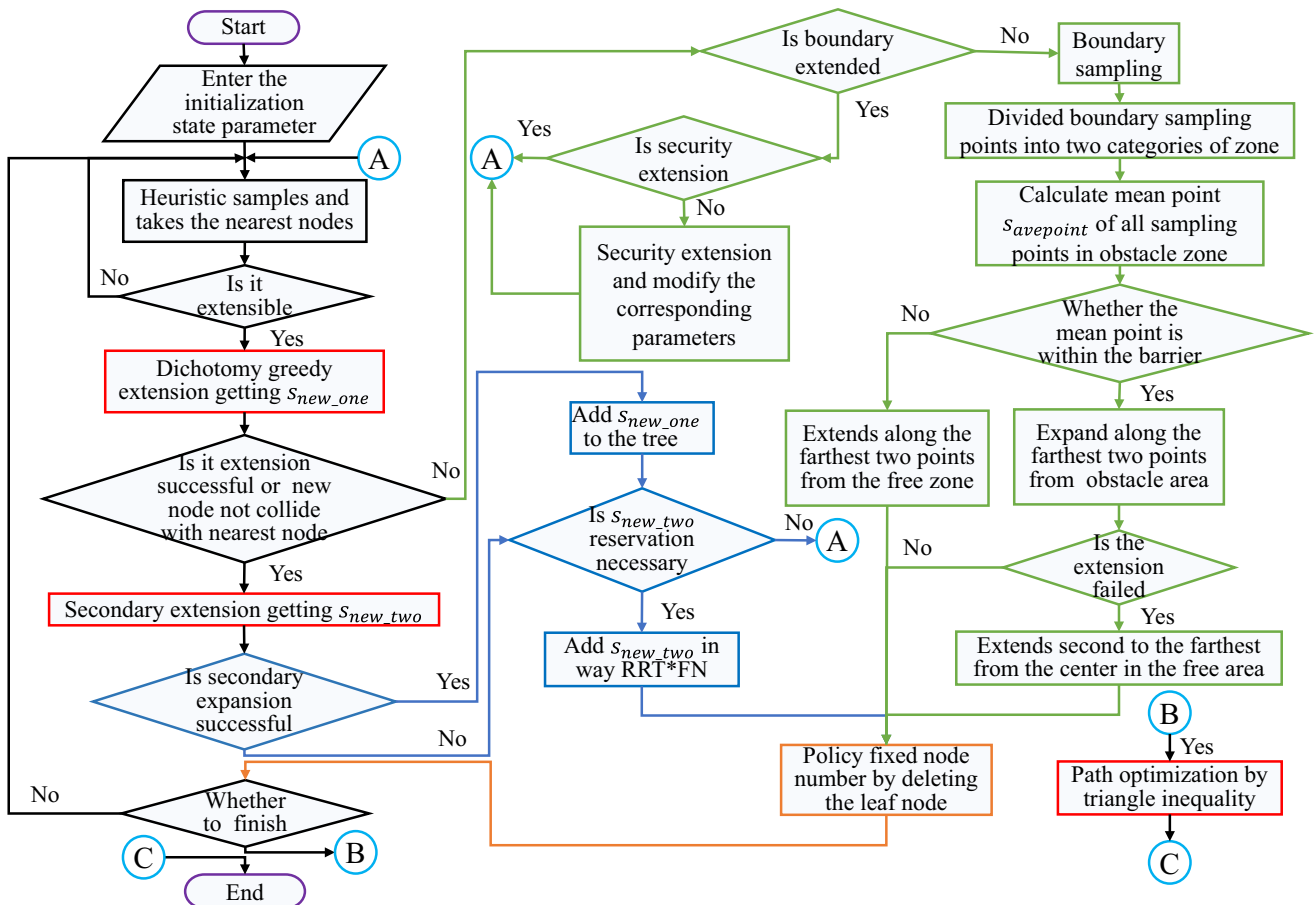
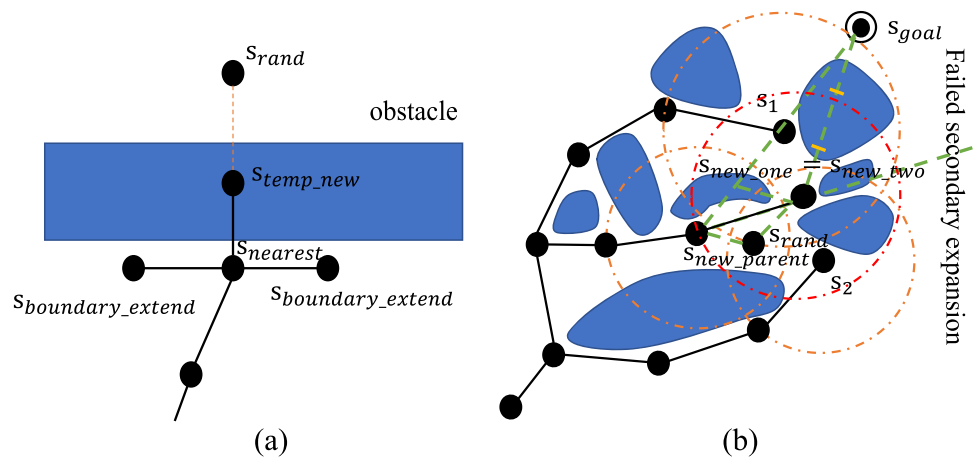
Manipulator path planning based on RRT*FN

To investigate manipulator path planning in a variety of complex scenarios, an improved RRT*FN [30] manipulator path planning algorithm that adapts to various scenarios is proposed, which is abbreviated as the vs.-RRT*FN algorithm for convenience of description. The pseudocode of the algorithm is shown in Algorithm 1.

In Algorithm 1, after the nearest node $s_{nearest}$ is obtained, the scalability of $s_{nearest}$ is judged through *ExtendabilityJudgment* to determine whether the algorithm should continue, as shown in Fig. 4a. The specific implementation steps are as follows: First, a temporary new point s_{temp_new} is expanded in the direction of a random point s_{rand} , and then, the collision between s_{temp_new} and $s_{nearest}$ is judged. If there is a collision, the boundary expansion flag attribute $s_{nearest}.Boundary_extended$ is judged. If the attribute is true, the boundary has been expanded, this cycle sampling expansion is discarded, and the unexpanded area expansion flag *Extend_to_unextended_areas_flag* is increased by

1. If the attribute is false, the algorithm enters the boundary sampling expansion process *BoundaryExtend*. The new node s_{new_two} obtained after the second expansion is not added directly to the tree based on the jurisdiction of the expanded node in the tree; that is, if any node to be expanded has no collisions with the expanded node and is in the spherical area with the expanded node as the center and the size of the distance between the extended node and the parent node as the radius, this node is considered an unnecessary extension point. Therefore, s_{new_two} uses *NodeNeedStorage* screening to prevent these unnecessary expansions. As shown in Fig. 4b, node s_{new_two} , whose secondary expansion is unsuccessful, is equivalent to s_{new_one} , which is under the jurisdiction of both s_1 and s_2 , and there is no obstacle between s_{new_two} and s_1 ; thus, in this case, s_{new_two} is an unnecessary extension node and should be discarded. Generally, the optimal path exists in an ellipsoid with the starting point position and goal point position as the two focal points. Memory consumption typically increases exponentially as the environment becomes more complex; therefore, RRT*FN limits the number of nodes in the tree based on the above algorithm to effectively save memory for other purposes in complex environments. Therefore, when the number of nodes reaches the maximum value, the forced removal of leaf nodes in the vs.-RRT*FN algorithm is determined by *ForcedRemoveNode*. Priority is given to the random selection of nodes to delete from the leaf node set outside the ellipsoid *Outellipsoid_leaf_nodes*; if the former nodes are not deleted, nodes from the leaf node set *Childless_nodes* are selected for deletion. The general process of the algorithm is shown in Fig. 5.

b Diagram of the process of filtering the unnecessary extension points, where the green dashed line represents the direction of the line between the nearest node and the goal point, the random point and the first extension point to form the first extension process



strategy. The blue boxes are the modified RRT*FN algorithm. The red boxes are the main improvements to the algorithm

Algorithm 1: VS RRT STAR FN (S_{init}, S_{goal})

```

1.  $T \leftarrow \text{InitializeTree}()$ 
2.  $T \leftarrow \text{InsertNode}(S_{init}, T)$ 
3.  $R \leftarrow \text{Maximum\_sampling\_space\_radius}$ 
4.  $T.R \leftarrow 0$ 
5. for  $i = 1$  to  $i = N$  do
6.    $S_{rand} \leftarrow \text{HeuristicSample}(R, T, R, S_{goal})$ 
7.    $S_{nearest} \leftarrow \text{Nearest}(T, S_{rand})$ 
8.    $\text{ExtendabilityJudgment}(S_{nearest})$ 
9.    $S_{new\_one\_flag} \leftarrow \text{Steer}(S_{nearest}, S_{rand}, S_{goal}, \text{Step\_to\_random}, \text{step\_to\_goal})$ 
10.  if  $\text{OutSampleSpace}(S_{new\_one})$  then
11.    continue
12.  end if
13.  if  $\text{flag}$  or  $\text{ObstacleFree}(S_{nearest}, S_{new\_one})$  then
14.     $S_{new\_two} \leftarrow \text{SecondExtend}(S_{new\_one}, S_{goal}, \text{Step\_to\_goal})$ 
15.    if not  $\text{EqualPoints}(S_{new\_two}, S_{new\_one})$  and not  $\text{ObstacleFree}(S_{nearest}, S_{new\_two})$  then
16.       $S_{new\_one\_Parent} \leftarrow S_{nearest}$ 
17.       $T \leftarrow \text{InsertNode}(S_{new\_one}, T)$ 
18.       $S_{nearest} \leftarrow S_{new\_one}$ 
19.    end if
20.    if not  $\text{NodeNeedStorage}(T, S_{new\_two})$  then
21.      continue
22.    end if
23.     $S_{near} \leftarrow \text{Near}(T, S_{new\_two})$ 
24.     $S_{new\_near} \leftarrow \text{ChooseParent}(S_{near}, S_{new\_two}, S_{nearest})$ 
25.     $T \leftarrow \text{InsertNode}(S_{new\_two}, T)$ 
26.     $T.R \leftarrow \text{UpdateExtendedArea}(T, T.R)$ 
27.     $R = \text{Distance}(S_{new\_two}, S_{goal})$ 
28.     $\text{Rewire}(T, S_{new\_near}, S_{new\_two})$ 
29.    if  $\text{ObstacleFree}(S_{new\_two}, S_{goal})$  and  $\text{Distance}(S_{new\_two}, S_{goal}) < \text{Step\_to\_random}$  then
30.       $\text{Reached\_nodes} \leftarrow \text{ReachedGoal}(\text{Reached\_node}, S_{new\_two})$ 
31.    else
32.      if  $\text{LeafNode}(S_{new\_two})$  then
33.         $\text{Childless\_nodes} \leftarrow \text{AppendNode}(\text{Childless\_nodes}, S_{new\_two})$ 
34.      end if
35.      if  $\text{OutEllipsoidLeafNode}(S_{new\_two})$  then
36.         $\text{Outellipsoid\_leaf\_nodes} \leftarrow \text{AppendNode}(\text{Outellipsoid\_leaf\_nodes}, S_{new\_two})$ 
37.      end if
38.    end if
39.    else
40.      if not  $S_{nearest}.\text{Boundary\_extended}$  then
41.         $\text{BoundaryExtend}(S_{nearest}, S_{goal})$ 
42.      else
43.         $\text{SecurityExtend}(S_{nearest}, S_{goal})$ 
44.         $\text{Extend\_to\_unextended\_areas\_flag} += 1$ 
45.         $R \leftarrow \text{ChangeGoalGuideRadius}(R, k * \text{Step\_to\_random})$ 
46.      end if
47.    end if
48.    if  $\text{NumberOfNode}(T) > M$  then
49.       $T \leftarrow \text{ForcedRemoveNode}(T, \text{Childless\_nodes}, \text{Outellipsoid\_leaf\_nodes})$ 
50.    end if
51.  end for
52.   $\text{Path} = \text{Connect}(T, \text{Reached\_nodes}, S_{goal})$ 
53.   $\text{Path} \leftarrow \text{RouteOptimize}(\text{Path}, T)$ 
54.  return  $\text{Path}$ 

```

Multiregion heuristic sampling strategy

There are several commonly used sampling methods, including uniform sampling, goal-biased sampling, and bounded sampling [31, 32]. Uniform sampling is a basic sampling scheme with uniform probability in the feasible configuration space; goal-biased sampling is based on a basic

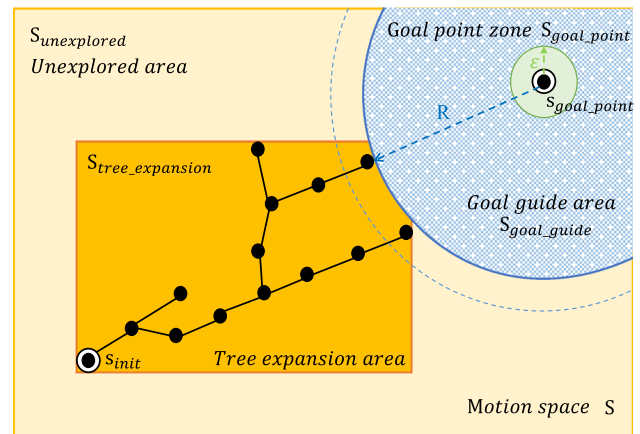


Fig. 6 Sampling space area division. The goal point area is shown in light green, the goal guide area is shown in blue, the expanded tree area is shown in orange, and the unexplored area is shown in yellow

sampling scheme that involves sampling goal configuration points with certain probabilities; and bounded sampling changes the sample boundaries either as the tree grows or based on specified conditions. Of the above three sampling methods, bounded sampling achieves the fastest response; however, the performance of a bounded sampling strategy in three-dimensional or higher dimensional environments has not been verified either mathematically or experimentally. Particularly in high-dimensional environments with complex obstacles, pure bounded sampling is often insufficient and may even result in higher costs than uniform sampling. Therefore, a combination of multiple sampling strategies should be considered. According to previous research [33], for completely random exploration, the relationship between the size of the area to be explored and the distance to the goal position is given by:

$$\lim_{n \rightarrow \infty} V_{\text{probe}} = c\pi R_{\text{probe_goal}}^3 \quad (2)$$

where n is the number of explorations, c is a constant, the initial state $R_{\text{probe_goal}}$ is the distance between the initial point and the goal point, and V_{probe} is the volume formed by the exploration points. The number of explorations increases exponentially with increasing exploration distance. Therefore, during the planning process, the main exploration area is dynamically reduced according to the goal point, reducing the exploration distance and greatly reducing the number of explorations.

Based on the above analysis, this paper proposes a multi-region heuristic sampling strategy based on a goal-oriented region. First, the entire sampling space is divided into four areas, namely, the goal guide area, the goal point area, the expansion tree area, and unexplored areas, as shown in Fig. 6. The main sampling area is the goal guide area, which is a sphere with the position of the goal point as its center and the closest distance between all nodes in the tree and

the goal point as its radius R . The radius R expands under certain conditions to enhance the environmental adaptability of the algorithm and reduce localized goal overheating. However, R generally gradually decreases to ensure that the algorithm can carry out the planning using as few sampling points as possible. The goal point area is located inside the sphere, with the goal point at its center and a small value ε as its radius. The sampling in this area further ensures that the goal can be rapidly located and introduces a goal bias characteristic to the algorithm. The *HeuristicSample* sampling strategy samples in the minimum circumscribed cube area of the tree, that is, the area where the tree is located. The purpose of this strategy is to gradually improve the spatial resolution in the tree, enhancing the path optimization. Because random expansions provide a path basis for the optimal path, it is necessary to sample in other unexplored regions, and to some extent, sampling in these regions can alleviate the temporary local goal overheating trap that may occur in goal-oriented region sampling.

Algorithm 2: *HeuristicSample* (R, T_R, S_{goal})

```

1.  $p_{rand} = \text{RandomNumber}(0, 1)$ 
2.  $S_{goal\_guide} = \text{AreaProduce}(S_{goal}, R)$ 
3.  $S_{goal\_point} = \text{AreaProduce}(S_{goal}, \varepsilon)$ 
4.  $S_{tree\_expansion} = \text{AreaProduce}(T\_R)$ 
5.  $S_{unexplored} = S - (S_{tree\_expansion} \cup S_{goal\_guide})$ 
6. if  $p_{rand} \leq P_{goal\_point}$  then
7.    $s_{rand} = \text{UniformSample}(S_{goal\_point})$ 
8. else
9.   if  $p_{rand} \geq P_{T\_R}$  then
10.     $s_{rand} = \text{UniformSample}(S_{tree\_expansion})$ 
11.   else
12.    if  $p_{rand} \leq P_{unexplored}$  or Equal(
13.      Extend_to_unextended_areas_flag, Maxib) then
14.       $s_{rand} = \text{UniformSample}(S_{unexplored})$ 
15.      Reset(Extend_to_unextended_areas_flag)
16.    else
17.       $s_{rand} = \text{UniformSample}(S_{goal\_guide})$ 
18.    end if
19.  end if
20.  Update( $P_{T\_R}$ )
21. return  $s_{rand}$ 
  
```

In Algorithm 2, the three parameters P_{goal_point} , $P_{unexplored}$ and P_{T_R} of the sampling strategy divide the $[0, 1]$ interval into four intervals, representing sampling in the goal area, the unexplored area, the goal guide area and the expansion tree area. The *Maxib* parameter indicates the maximum cumulative number of invalid boundary extension nodes. When the number of invalid boundary extension nodes reaches the *Maxib* value, additional unexplored areas are sampled. This method further eliminates invalid boundary point expansions; thus, this strategy reduces the probability that the nearest neighbor node under the current sampling is determined as a boundary point with boundary expansion. The value of P_{goal_point} is generally small and remains unchanged throughout the process, and the value of $P_{unexplored}$ is also small. At the beginning of the planning stage, the value of

P_{T_R} is set to a large value to ensure that the algorithm explores the goal guidance area; then, this value gradually decreases as the number of nodes increases during the planning process until it reaches a stable value.

Greedy expansion based on dichotomy and double expansion

During the node expansion process, new nodes are generated not by expansion in a random direction or by simultaneously expanding in the directions of the goal point and a random point with a fixed step size but rather by expanding in the directions of the variably gravitational forces of the goal and random points. Each new node is generated as a result of this joint action, and the weight values of both forces are dynamically adjusted by the dichotomy method to adapt to various environments, where the dynamic weight value is based on the distance between $s_{nearest}$ and s_{rand} , and the length is bisected each time to test whether there are obstacles between $s_{nearest}$ and s_{new_one} . Finally, an effective first expansion point is formed to accelerate the convergence regardless of the scene. To further accelerate the convergence process, the newly generated node expands again toward the goal position with a step size of the average diameter of the narrow channel in the environment. The above process is shown in Fig. 7. Due to the wide range of potential scenes, it is impossible to depict all situations in diagram form; thus, only typical and generic situations are described in this work.

Algorithm 3: *BoundaryExtend* ($s_{nearest}, S_{goal}$)

```

1.  $S_{sampling\_points} \leftarrow \text{LocalSampling}(s_{nearest})$ 
2.  $S_{loc\_obs}, S_{loc\_free} \leftarrow \text{DividePointsIntoBarriersAndFreeZones}(S_{sampling\_points})$ 
3.  $S_{second\_free} \leftarrow \text{FindSecondPointFromNearest}(S_{loc\_free}, s_{nearest})$ 
4.  $s_{avepoint} \leftarrow \text{Mean}(S_{loc\_obs})$ 
5. if ObstacleInside( $s_{avepoint}$ ) then
6.    $s_{obs1}, s_{obs2} \leftarrow \text{FurthestPairPoints}(S_{loc\_obs})$ 
7.   if  $s_{nearest}, \text{Parent}, \text{Boundary\_extended}$  then
8.      $\text{Exp\_fail\_flag} \leftarrow \text{ExpandAwayNearestParentAlongPoints}($ 
9.        $s_{obs1}, s_{obs2}, s_{nearest})$ 
10.    if Exp\_fail\_flag then
11.       $s_{away} \leftarrow \text{FurthestPointAwayNearestParentNode}($ 
12.         $s_{second\_free}, s_{nearest})$ 
13.      ExpandOnePoint( $s_{away}, s_{nearest}$ )
14.    end if
15.  else
16.    Expand\_success\_flag  $\leftarrow \text{ExpandBidirectionAlongTwoPoints}(s_{obs1}, s_{obs2})$ 
17.    if not Expand\_success\_flag then
18.      ExpandAllPointsInSecondFreeSpace(
19.        TakePoint( $S_{second\_free}, s_{nearest}$ ))
20.    end if
21.  end if
22. else
23.   if Distance( $s_{avepoint}, s_{nearest}$ )  $\geq \lambda$  then
24.     ExpandTowardsAvepointobs( $s_{avepoint}, s_{nearest}$ )
25.   else
26.      $s_{free1}, s_{free2} \leftarrow \text{FurthestPairPoints}(S_{loc\_free})$ 
27.     ExpandTowardsGoalAlongPoints( $s_{free1}, s_{free2}, S_{goal}$ )
28.   end if
29. end if
  
```

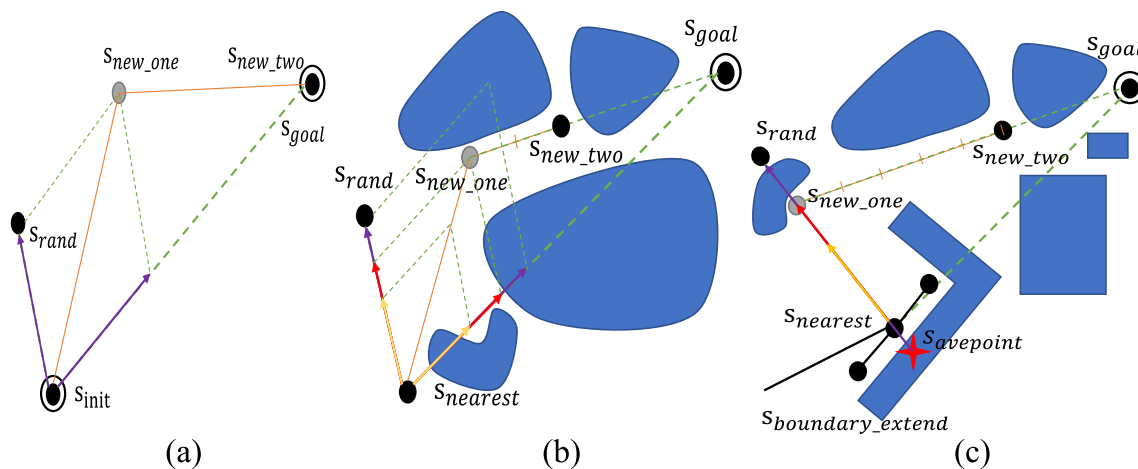


Fig. 7 Dichotomy method greedy extension and twice extension. **a** Barrier-free environment. **b** No-boundary expansion point in an environment with obstacles. **c** Boundary extension point in an environment with obstacles

When the nearest-neighbor node is a boundary point that has been expanded by *BoundaryExtend*, because the optimal choice is to leave the boundary area as often as possible, in this case, the new node is expanded in a direction away from obstacles, which s_{avepoint} points in the direction of the nearest-neighbor node, whose method named security extension policy. In the *Steer* routine, if the dichotomy method successfully generates a new point, the new point and a true flag are returned. In contrast, if s_{nearest} still collides with s_{new} after multiple dichotomies, the new point and a false flag are returned until the one-way expansion length is essentially the same as *Step_to_random*; then, new points are generated by simultaneously extending s_{nearest} toward s_{rand} with a length of *Step_to_random* and toward s_{goal} with a length of *Step_to_goal*.

Boundary extension based on local environment sampling

In the planning process, if the normal RRT*FN expansion fails, that is, when the nearest neighbor node is considered a boundary point without boundary expansion, the algorithm enters the local environment sampling boundary expansion to determine a new method for crossing the narrow channel. The pseudocode of this process is summarized in Algorithm 3.

In Algorithm 3, boundary expansion is carried out by locally sampling the boundary points with the sampling function *LocalSampling*. The purpose of sampling at the boundary points is to determine a basis for boundary expansion. Therefore, the collected samples must satisfy the absolute symmetry of the s_{nearest} sampling center point, and each dimension is distributed in both directions to ensure that narrow channels in any direction can be detected. To easily obtain enough sampling points to describe a local

structure, the *LocalSampling* sampler first takes s_{nearest} as the center, and each dimension coordinate of this center is $\pm \text{Step_to_random}$, forming a sampling point set in which the number of sample points is twice the number of s_{nearest} dimensions. The previous sampling method is then repeated with these sampling points as the center. However, in this case, each dimension coordinate is $\pm \text{Step_to_random}/2$. The goal of the sampler is to take a point as the center and radiate equal lengths to each dimension to obtain the corresponding point set. This local sampling method dynamically and automatically increases the number of sampling points as the spatial dimension increases. After the sampling point set $S_{\text{sampling_points}}$ is determined, the points are divided into a free zone point set $S_{\text{loc_free}}$ and an obstacle zone point set $S_{\text{loc_obs}}$. Then, the mean point s_{avepoint} of all points in the obstacle zone is calculated. As shown in Fig. 8a, if the mean point is within the obstacle zone, the new point is expanded in either one or both directions along the line consisting of the two farthest points, s_{obs1} and s_{obs2} . If the parent node of s_{nearest} is a boundary point that has been expanded by *BoundaryExtend*, the point expands in the direction away from the parent node of s_{nearest} along the line connecting the two points. If the above expansion fails, a step-shaped L-type obstacle is most likely encountered, as shown in Fig. 8b. At this time, the point in $S_{\text{second_free}}$ is taken as the new point, and the principle is still based on the parent node attribute of s_{nearest} . If the mean point is in the free zone and its distance from s_{nearest} is greater than a threshold λ that is slightly smaller than *Step_to_random*, boundary point s_{nearest} lies at the entrance outside the narrow passage, and the new point is directed to expand in the direction of mean point s_{avepoint} . Otherwise, the boundary point lies in the narrow channel, as shown in Fig. 8c. In this case, s_{nearest} expands along the direction toward the goal point of the two-point line generated by

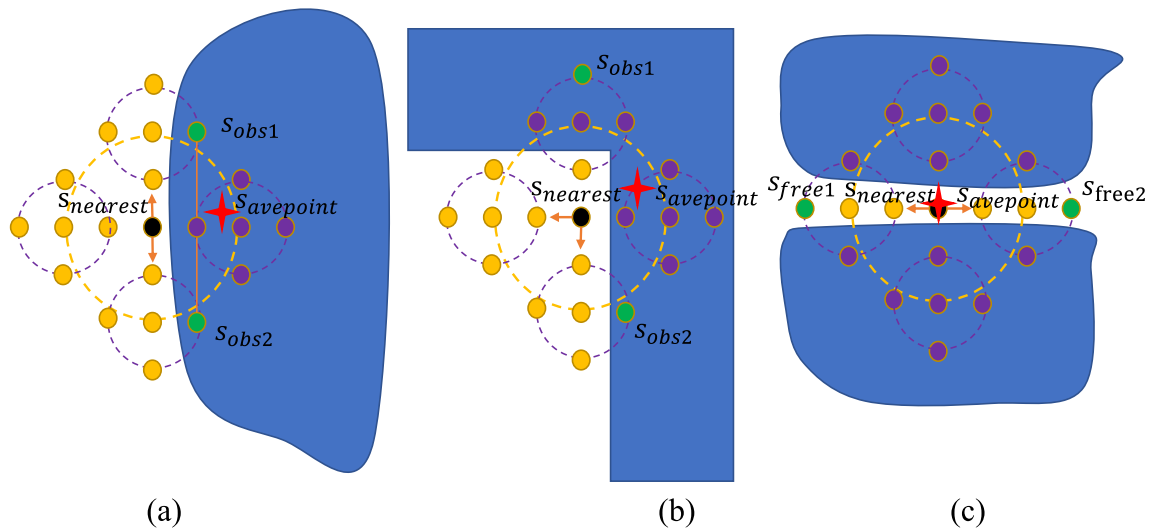
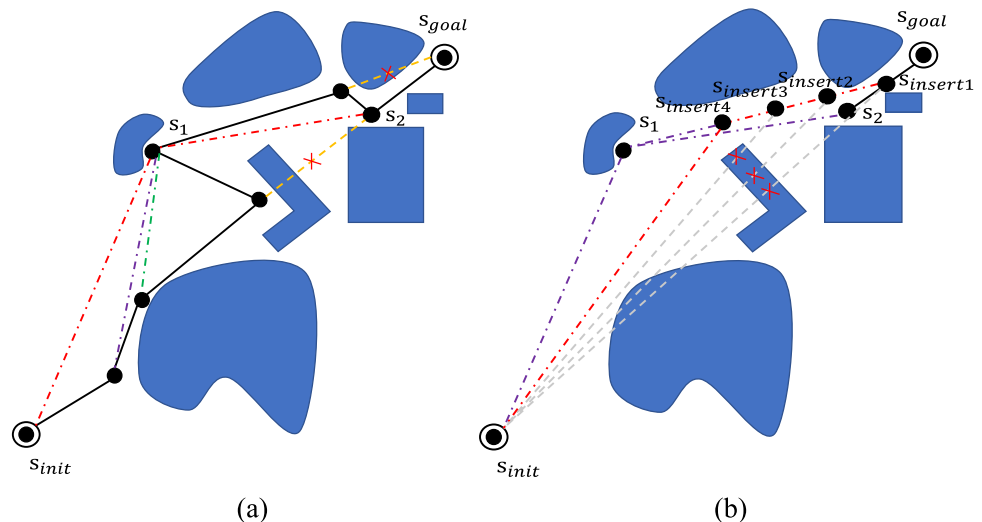


Fig. 8 Local environmental sampling at the boundary points. **a** Square-type obstacle. **b** Step-shaped L-type obstacle. **c** Synthetic narrow channel-type obstacles

Fig. 9 Path optimization based on a triangle inequality.

a Original triangle inequality optimization path. **b** Node insertion into the triangle inequality optimization path



Furthest Pair Points in the free zone. The main local environmental sampling cases are shown in Fig. 8. In the above process, the area T_R where the expansion tree is located and the radius R of the goal guide area must be maintained because they are included in each *Expand* process.

Path optimization based on a triangular inequality

The node cost is the basis of the parent node reselection, rewriting and path optimization algorithms [34]. In this study, the trade-off between the advantages and disadvantages of node extension, including the path cost, is assessed using the following heuristic value function:

$$\text{Cost}(s_i) = k_1 \cdot P(s_i) + k_2 \cdot \frac{1}{F(s_i)} + k_3 \cdot T(s_i) \quad (3)$$

where $P(s_i)$ is the actual path cost of point s_i , that is, the total step length from the starting point to point s_i ; $1/F(s_i)$ is the safety cost, where the value of $F(s_i)$ is the average distance between the location of s_i and nearby obstacles; $T(s_i)$ is the stability cost, which is the average change rate of each robotic arm joint as the parent node of s_i moves to point s_i ; and k_1 , k_2 and k_3 are coefficients of the three subcosts that represent their proportion in the total cost.

After the final path is obtained using the proposed algorithm, the path is further optimized. A schematic diagram of this process is shown in Fig. 9. For ease of description, only the actual node path cost is considered here. The path optimization step can be approximately described according to the goal point of the final path. If the grandparent node of s_i acts as the parent node of s_i , the cost of s_i is reduced without collisions; that is, when the triangle inequality rule

is satisfied, the parent node of s_i is adjusted to the grandparent node of s_i . Otherwise, if a collision occurs between the grandparent node of s_i and s_i , which means that the s_i node has been optimized, the parent node s_i is optimized, and so on until the initial node s_{init} is the node to be optimized, thus completing the optimization process. After the optimization process, the corresponding points on the newly obtained path can be added to repeat the above triangle inequality optimization process. For example, the algorithm takes the goal point as the base point and walks a certain distance along the goal point in the s_2 direction to form the corresponding insertion point. The distance takes the minimum value, which is the average radius of all obstacle volumes when expressed as a ball and the half length of the line between the goal point and s_i , after which the previous optimization process is repeated from that insertion point (all of the above operations include robot linkage collision detection; see the next section for details). However, this process increases the time cost. To balance path optimization and real-time performance, this step is not implemented in the proposed algorithm.

Collision judgment of robot ontology and coupling of node selection

The Flexible Collision Library (FCL) [35] can be used to detect collisions between rigid, deformable, or articulated objects and point clouds or octree graphs, although precise modeling is not required during collision detection. However, the FCL input model needs to be sufficiently accurate. For the robotic arm obstacle avoidance problem, it usually takes too much time to model the robotic arm and obtain the point cloud information of the scene. At the same time, due to the influence of environmental noise and shooting conditions, the collected point cloud requires considerable preprocessing work. Therefore, considering the algorithm development process and computation to facilitate real-time collision detection, the direct modeling of manipulators and obstacles is simplified based on the idea of bounding boxes and spatial superposition [36] to simplify the modeling of manipulators and obstacles directly.

Algorithm 4: *ObstacleFree* (s_1, s_2)

```

1.  if CollisionJudgment ( $s_1, s_2$ ) then
2.    return 0
3.  else
4.     $S_{pose\_collection} \leftarrow$  Take K+1 corresponding points from the line
      segment  $s_1s_2$  include  $s_1$  and  $s_2$ .
5.    count = 0
6.    for  $s_i, s_{i+1}$  in  $S_{pose\_collection}$  do
7.      Eight Groups of Robot Inverse Kinematics States:  $\theta[8] \leftarrow$ 
        InverseKinematics( $s_{i+1}$ )
8.       $\theta'[8] \leftarrow$  Sort  $\theta[8]$  according to  $\sum_{j=1}^6 |\theta_{s_{ij}} - \theta_{s_{i+1,j}}|$  rule
        from small to large
9.      flag1 = 0
10.     for n=0 to n=7 do
11.        $\theta = \theta'[n]$ 
12.       Link[6]  $\leftarrow$  obtain the state of each link from each joint  $\theta$ 
13.        $S_{Link} \leftarrow$  the endpoints of the simplified model of each
        connecting rod are obtained from the model
        in Fig. and Link[6].
14.       flag2 = 0
15.       for m=0 to m=5 do
16.         if CollisionJudgment( $S_{Link}[m], S_{Link}[m+1]$ ) then
17.           flag2 = 1
18.           break
19.         end if
20.       end for
21.       if flag2 == 1 then
22.         continue
23.       else
24.         flag1 = 1
25.         break
26.       end if
27.     end for
28.     if flag1 == 0 then
29.       return 0
30.     else
31.       if all not CollisionJudgment( $S_{Link}[a], S_{Link}[a+1]$ )( $\theta_{s_i}$  to
         $\theta_{s_{i+1}}$ , a=0 to a=5) then
32.         count += 1
33.       end if
34.     end if
35.   end for
36.   if count == K then
37.     return 1
38.   else
39.     return 0
40.   end if
41. end if

```

In practical engineering applications, because obstacles typically have irregular shapes, they are difficult to accurately model mathematically. Thus, collision judgment usually adopts a spatial geometric envelope method to simplify the obstacle and manipulator model. In this study, cylinders are

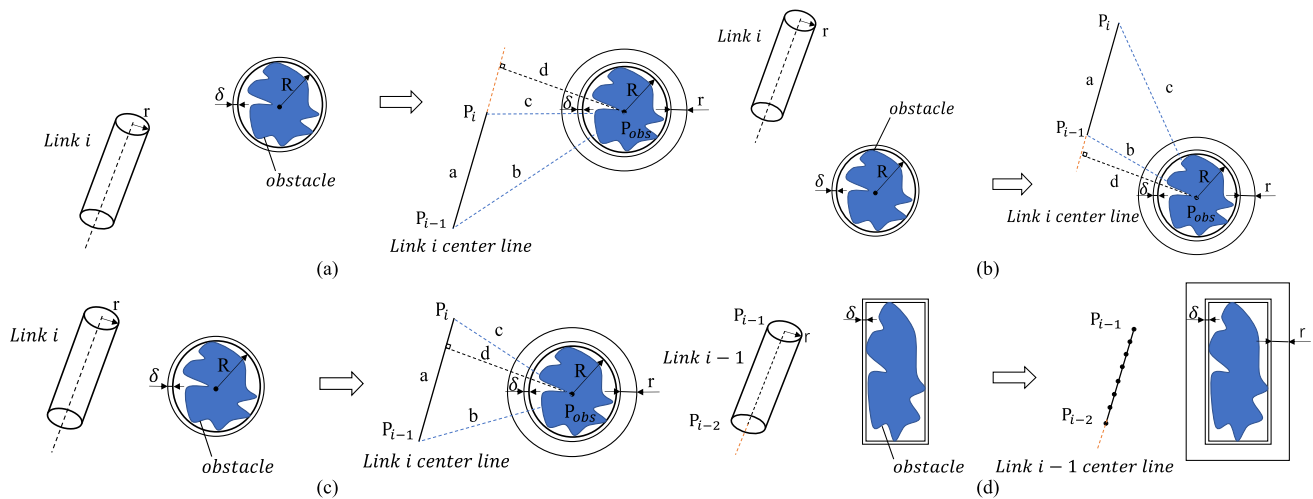


Fig. 10 Collision judgment model. **a–c** Sphere envelope obstacle and manipulator. **d** Square envelope obstacle and manipulator

used to approximate manipulator links. Because obstacles can have different shapes, modeling obstacles as enclosed spheres may lead to large deviations, which is not conducive to accurate representation of the space. Therefore, an envelope sphere or a cube (cuboid or cube) is chosen to approximate obstacles. For the spherical envelope obstacle shown in Fig. 10a–c, under the ball envelope obstacle condition, the radius of the cylindrical model of the connecting rod is r , the radius of the obstacle envelope ball is R , and the distance from the center of the envelope ball to the centerline of the connecting rod is d , $a = \|P_i \vec{P}_{i-1}\|$, $b = \|P_{obs} \vec{P}_{i-1}\|$, and $c = \|P_{obs} \vec{P}_i\|$. Then, the following conclusions can be drawn: when $a^2 + b^2 \leq c^2$ as in Fig. 10a, the closest distance between the connecting rod and the envelope ball is $b - R - r - \delta$; when $a^2 + c^2 \leq b^2$ as in Fig. 10b, the closest distance between the connecting rod and the envelope ball is $c - R - r - \delta$; when the relationship between a , b and c is expressed as shown in Fig. 10c, the closest distance between the connecting rod and the envelope ball is $d - R - r - \delta$, where the distance d can be expressed using a , b , and c and can be solved by: $d = \frac{2\sqrt{l(l-a)(l-b)(l-c)}}{a}$, $l = (a + b + c)/2$. In summary, the rules of collision detection between the connecting rod and obstacle under the envelope sphere model are as follows:

$$\text{Let } D = \begin{cases} b - R - r - \delta, & a^2 + b^2 \leq c^2 \\ c - R - r - \delta, & a^2 + c^2 \leq b^2 \\ d - R - r - \delta, & \text{other} \end{cases}$$

robot arm link collides with an obstacle. For collision evaluation between a cuboid envelope box and the manipulator, as shown in Fig. 10d, if the centerline segment $P_{i-1}P_{i-2}$ of link $i-1$ is outside the obstacle envelope box, there is no collision between link $i-1$ and the obstacle; otherwise, there is a collision. The spatial relationship between the line segment

$P_{i-1}P_{i-2}$ and the envelope box during the cube envelope box collision evaluation process is difficult to determine directly; however, it is relatively simple and convenient to determine whether a point is located in a certain space. The line segment $P_{i-1}P_{i-2}$ can be sampled at a certain length to determine the corresponding points on the line segment. Thus, only the spatial relationship between these points and the obstacle envelope box must be determined. If all sampling points are outside the envelope box, the line segment $P_{i-1}P_{i-2}$ is outside the obstacle envelope; otherwise, the line segment is within the obstacle envelope. If the obstacle and each link of the manipulator satisfy the no-collision condition, the current state of the manipulator satisfies the no-collision condition.

This study embeds collision detection into the planning algorithm, which is more scalable than the FCL encapsulation interface. Since obstacle models can be very complex in real situations, using a single sphere/cube (cuboid or cube) to simplify the model may make the detection results unreliable. In this case, an obstacle can be modeled as a superposition of multiple spheres/cubes (cuboid or cube) and solved based on the method proposed above. Therefore, only the simplified part of the obstacle model needs to be changed. This situation will be addressed in future work.

Algorithm 4 shows the manipulator ontology obstacle avoidance rules. First, it is determined that the end effector does not collide with obstacles during its movement from s_1 to s_2 , and then the algorithm judges whether each link collides during the movement of the end effector along the known collision-free path s_1s_2 . The connecting rod collision judgment rule is determined according to the model shown in Fig. 8, and the $\text{CollisionJudgment}(S_{\text{Link}}[a], S_{\text{Link}}[a + 1])(\theta_{s_i} \text{ to } \theta_{s_{i+1}}, a = 0 \text{ to } a = 5)$ statement indicates that the

manipulator operates from joint state θ_{s_i} to $\theta_{s_{i+1}}$ and judges the collision of six connecting rods at each moment.

Time complexity analysis

The time complexity of the proposed algorithm is determined from Algorithm 1, as the time required to add N nodes to the tree (ignoring initialization) can be calculated as the sum of the $C_1 N$ iterations of lines 5–21, the N iterations of lines 22–49 and the $C_2 N$ iterations of lines 50–51 in Algorithm 1 (where C_1 and C_2 are constant positive integers):

$$T(N) = T_{\text{sample}}(C_1 N) + T_{\text{nearest}}(C_1 N) + T_{\text{extend}}(N) + T_{\text{add}}(N) + T_{\text{path_optimize}}(C_2 N) + T_{\text{get_path}}(C_2 N) \quad (4)$$

here, T_{sample} , T_{nearest} , T_{extend} , T_{add} , $T_{\text{path_optimize}}$, and $T_{\text{get_path}}$ correspond to lines 5–51, representing sampling, nearest node fetching, tree expansion, node addition, path optimization, and path extraction, respectively. T_{sample} , T_{add} , $T_{\text{path_optimize}}$ and $T_{\text{get_path}}$ are linear operations for the total number of nodes and can be completed in linear time, so their complexity is $O(N)$. Tree extension T_{extend} takes considerable time because of the existence of collision detection and intelligent expansion strategies, but the time is not determined by the number of nodes in the tree, and the extension can still be completed in linear time. The proposed algorithm does not further optimize the nearest node traversal lookup strategy, nor does it use a better data structure such as a kd-tree to store the tree, but directly adopts various strategies to store as few nodes as possible in the tree, so the highest overall complexity is still that of T_{nearest} . Each time the nearest node is obtained, the distance between each node in the tree must be calculated, so T_{nearest} can be expressed as:

$$T_{\text{nearest}} = \sum_{i=1}^{C_1 N} (i-1) T_{\text{dist}} = \frac{C_1^2 N^2 - C_1 N}{2} T_{\text{dist}}, \quad (5)$$

where T_{dist} denotes the time required to calculate the distance between any two points, and the time complexity approximates $O(1)$. Therefore, the time complexity of nearest node acquisition is $O(N^2 - N)$. In summary, the total time complexity of the proposed algorithm is:

$$O(N) + O(C_1 N^2 - N) + O(N) + O(N) + O(N) + O(N) \approx O(N^2). \quad (6)$$

Experimental results

To verify the real-time performance, effectiveness and path optimization performance of the proposed algorithm, corresponding simulations and comparative experimental analyses are carried out. In this study, the algorithms and simulations were carried out on the same equipment, which was configured with an Intel i7-9750H model CPU with a frequency of 2.60 GHz and 16 GB of memory.

Two-dimensional simulation analysis

A two-dimensional space requires a small amount of computation and produces relatively intuitive results. Thus, in this section, four algorithms are simulated in a two-dimensional space to evaluate the performance of the proposed algorithm. The performance of the RRT, RRT*, RRT*FN and vs.-RRT*FN algorithms is analyzed and evaluated by comparing the simulation results in various environments. Each algorithm is simulated in two complex environments in the two-dimensional space. In all cases, the total area of the movement space is 500 mm $\times$ 300 mm (dimensions in the map are in cm, here converted to mm). The gray areas in the map represent the boundaries of the movement space that are also considered obstacles, and the pink areas represent obstacles. In scene 1, the starting point is (20, 20), the goal point is (490, 240), the general step size $Step_to_random$ is 8, $Step_to_goal$ is 4, P_{goal} is 0.1, and the initial values of P_{T_R} and P_{unexpend} are 0.95 and 0.2, respectively. The fixed maximum number of nodes is 2000, and the maximum number of iterations is set to 10,000. The algorithm planning result is shown in Fig. 11. The blue line represents the exploration tree, and the red solid line represents the path planned by the corresponding algorithm. To reflect the boundary expansion characteristics, the algorithm does not add a safety distance in the actual collision judgment; thus, the path line appears to be very close to the obstacles.

The RRT, RRT*, RRT*FN and proposed algorithm planning diagrams are presented in Fig. 11, which shows that the RRT and RRT* algorithm planning diagrams are similar to the RRT*FN planning diagram. The search time and path length used in the planning of these four algorithms are considered in the difference comparison to verify the performance of the proposed algorithm. The statistical results are shown in Fig. 12. A total of 21 groups of experiments were conducted, with an average of 100 experiments per group. A comparison of the graphs shows that the proposed algorithm planning time remains low even for complex environments. The searched path remains essentially near the line connecting the starting point and the goal point even if the path is not optimized, with the path length further reduced after optimization. The figure clearly shows that the vs.-RRT*FN

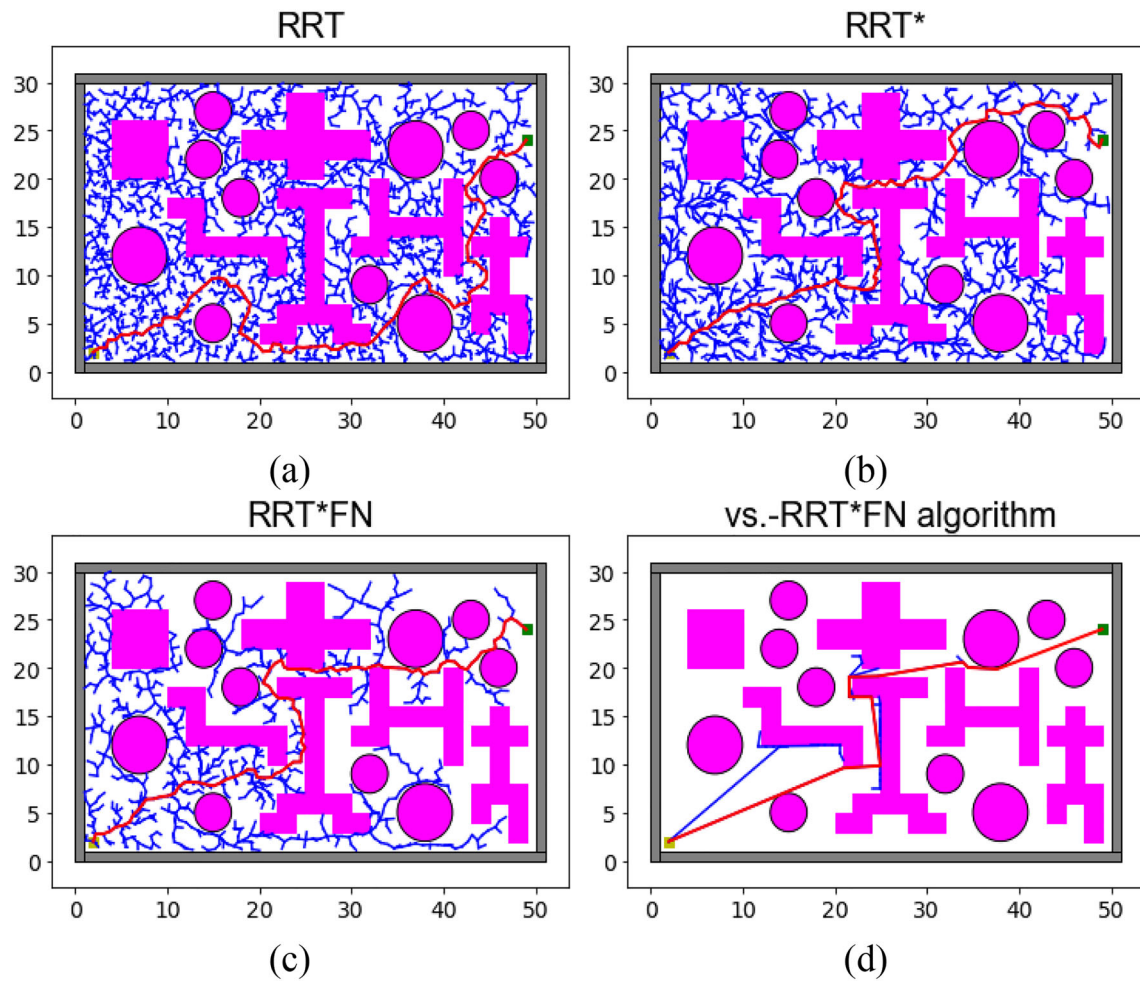


Fig. 11 Algorithm results for scenario 1. **a** RRT; **b** RRT*. **c** RRT*FN. **d** vs.-RRT*FN

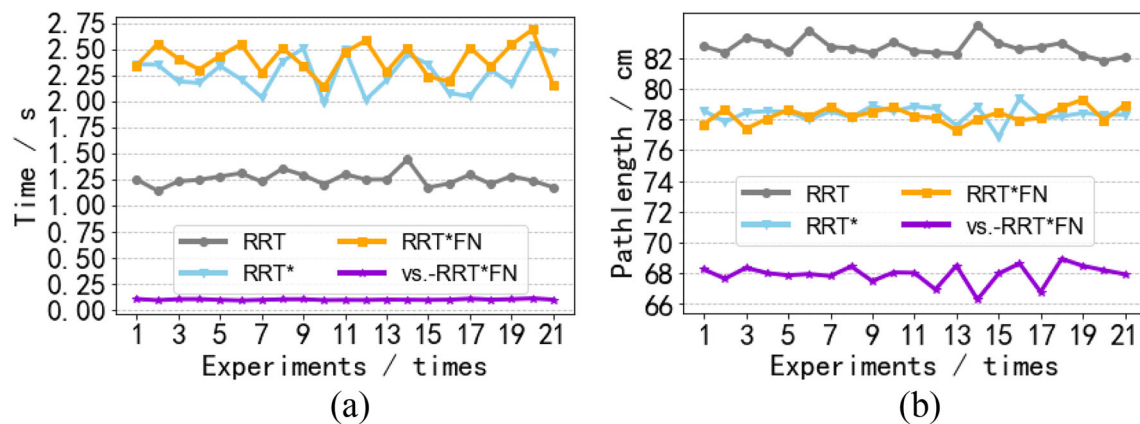


Fig. 12 Algorithmic planning performance comparison. **a** Search time. **b** Path length

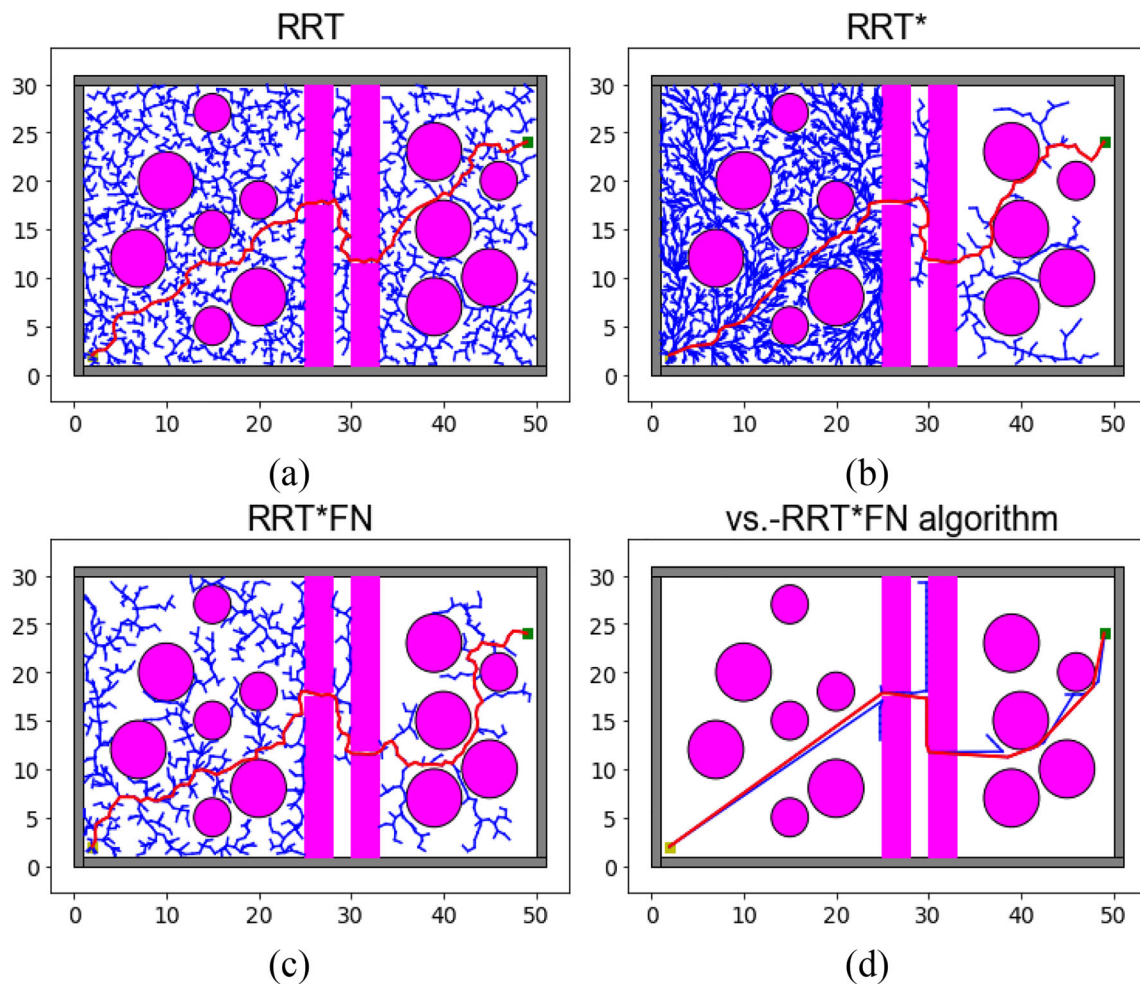


Fig. 13 Algorithm results for scenario 2. **a** RRT. **b** RRT*. **c** RRT*FN. **d** vs.-RRT*FN

algorithm achieves a shorter path length and faster convergence than the other three algorithms.

The proposed algorithm can adapt to a variety of scenarios. Scenario 1 represents a complex non-narrow-channel environment. Next, the algorithm was applied to a complex environment with narrow channels in scenario 2, and the basic settings were the same as in scenario 1. The RRT, RRT*, RRT*FN, and proposed algorithm visualization results are shown in Fig. 13, and a comparison of the performance index of the four algorithms is shown in Table 2. The data in Table 2 were obtained by averaging 1000 scenario 2 experiments for each algorithm. As shown in Table 2, the parameters used to represent the performance of the algorithm are the average running time of the algorithm, the average path length, and the success rate. In general, the average run time and the average path length are calculated as the average in all cases where the path is successfully found. The so-called search success refers to the total number of iterations in 10,000 to find a collision-free path.

Table 2 Comparison of the algorithms in scenario 2

Algorithm	Average run time (s)	Average path length (mm)	Success rate (%)
RRT	1.40	731.00	79
RRT*	2.67	716.50	77
RRT*FN	1.93	725.60	72
vs.-RRT*FN	0.09	661.60	100

There is no obvious relationship between the three. In general, when the search is successful, the longer the search time is, the shorter the path length that can be optimized. Comparative analysis shows that the proposed algorithm has a very high planning success rate of nearly 100% in the narrow-passage environment. Furthermore, the proposed algorithm consumes less time; specifically, the proposed algorithm has a better path and a higher planning speed. Based on the above experimental analysis, the proposed algorithm can ensure

Table 3 Results of experiments in three dimensions

Scenario	Algorithm	Average run time (s)	Success rate (%)
Complex environment	RRT*FN	11	98
	vs.-RRT*FN	4	100
Narrow channel	RRT-Connect	47	72
	vs.-RRT*FN	16	99

effective, real-time path optimization performance in a variety of complex environments.

Experiments in three dimensions and ROS

A 3D obstacle avoidance environment was constructed in Python, and the proposed algorithm was compared with the standard RRT*FN [13] and RRT-Connect [23] algorithms in 3D simulations. To test the adaptability of the proposed algorithm to complex and narrow passages in a three-dimensional environment, a small ball with a radius of 2 cm is set as the navigation object, the safety distance delta is set to 0.2 cm, the entire space layout is 100 cm $\times$ 100 cm $\times$ 100 cm, and the minimum gap between narrow passages is 5 cm. A total of 1000 sets of experiments were carried out in each environment to obtain the results shown in Table 3. The shortest distance between the final path and the obstacle was 0.3 cm. Figure 14a and c shows that the search trees obtained using the RRT*FN and RRT-Connect algorithms fill nearly the entire space, while the proposed algorithm is more accurate and effective at using sampling points due to the heuristic sampling algorithm. Furthermore, the combination of the greedy expansion of the bisection method and the twice-expansion strategy causes the tree to converge to the goal point position more rapidly while producing the optimal expansion process path. In particular, a special and effective boundary expansion method is used for the boundary points.

Finally, the path is optimized based on the triangle inequality. Thus, the proposed algorithm not only ensures real-time performance but also generates better paths than the other algorithms in various scenarios.

In this section, to verify the feasibility of the manipulator path planning method described in this paper, the proposed algorithm is applied to a six-DOF serial manipulator controlled by the ROS platform, and the preset hyperparameters, such as safe distance and extended step, are consistent with the previous section.

As shown in Fig. 15, the path planning algorithm guides the manipulator to complete the movement from the starting position to the designated position on the first floor of the shelf for storing workpieces, as shown in Fig. 15a–d. After the manipulator leaves the first floor, the manipulator continues to move to the designated position on the second floor of the shelf, as shown in Fig. 15e–h. Finally, the manipulator safely returns to the end point in Fig. 15j, as shown in Fig. 15i–j. The above is called part sorting task T. In the diagram, the orange–red area indicates that the shelf also acts as an obstacle, and a large number of dark green additional obstacles are densely distributed around the robotic arm. Figure 16 shows the result of the improved RRT_Connect [22] controlling the robotic arm to complete task T. Figure 17 presents the distance between each link and the end effector and the nearest object that the robot may collide with during the completion of task T. Figure 18 shows the changes for each joint of the manipulator during the operation of the two. The change rate of each manipulator joint using vs.-RRT*FN is smaller and more stable. For task T, RRT*, RRT*FN, RRT_Connect and the proposed manipulator control algorithm were used for 500 experiments, providing the data presented in Table 4. During the whole planning process to complete task T, the manipulator operation time should not exceed 80 s; otherwise, the planning was deemed invalid. An examination of the data presented in Table 4 shows that the proposed algorithm is more efficient in planning, and each test is successful.

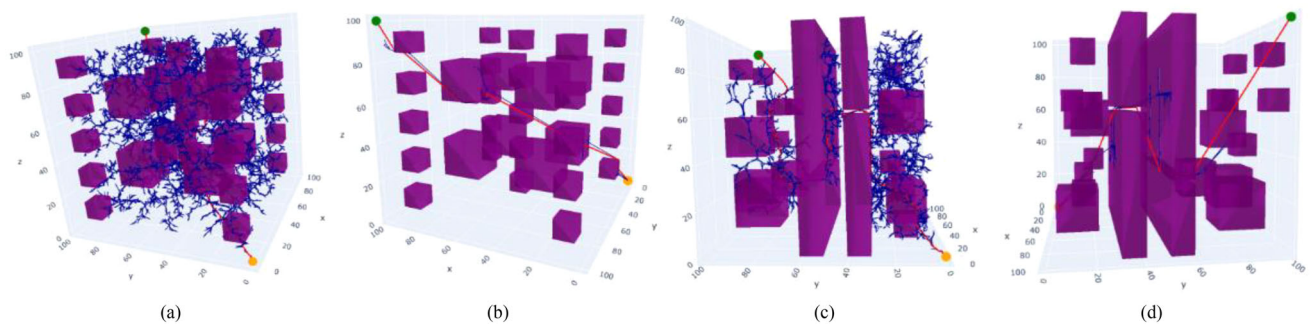


Fig. 14 Path planning results in a 3D environment; the yellow point is the start point, and the green point is the end point. **a** RRT*FN in a complex environment. **b** The proposed algorithm in a complex

environment. **c** RRT-Connect in a narrow-channel environment. **d** The proposed algorithm in a narrow-channel environment

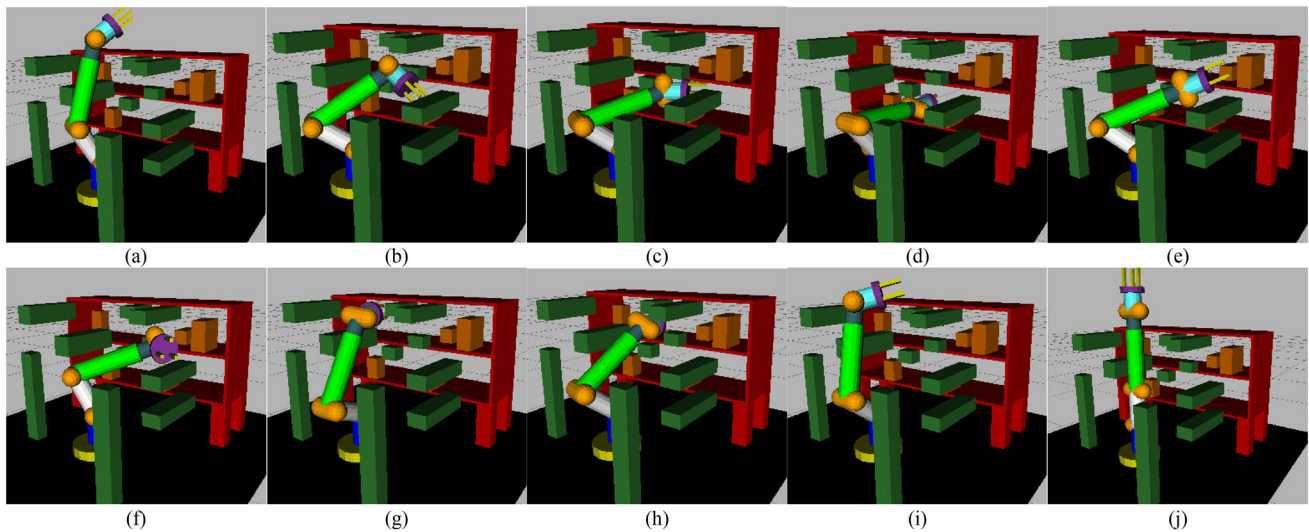


Fig. 15 Six-DOF robotic arm simulation using vs.-RRT*FN based on the ROS. **a–e** Complete bottom shelf pickup action. **f–i** Complete upper shelf placement action. **j** Initial state and termination position

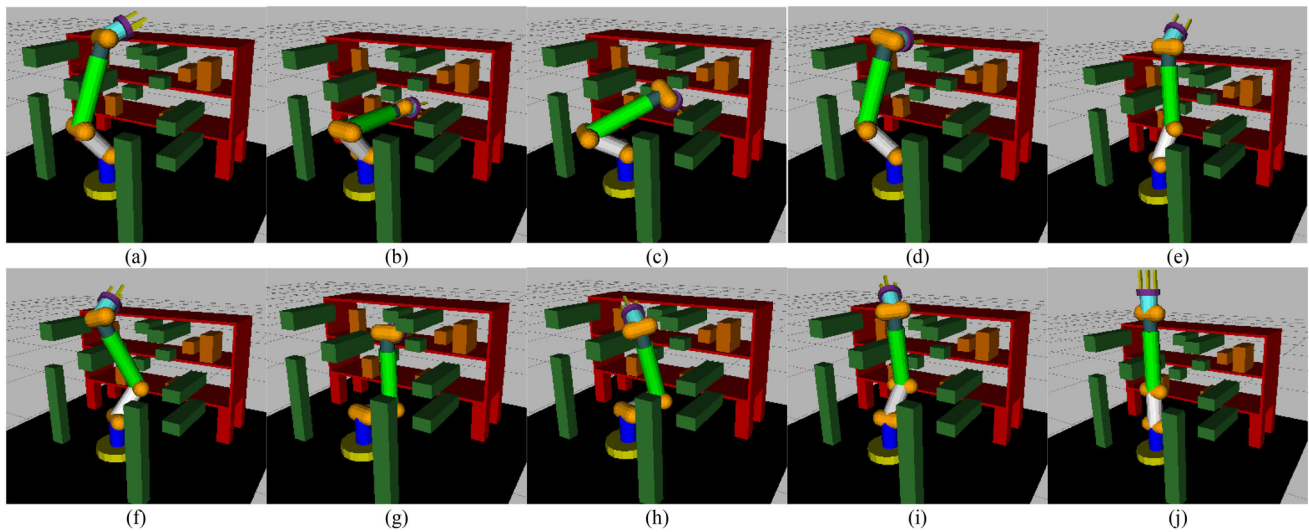


Fig. 16 Six-DOF robotic arm simulation using RRT_Connect based on the ROS. **a–e** Complete bottom shelf pickup action. **f–i** Complete upper shelf placement action. **j** Initial state and termination position

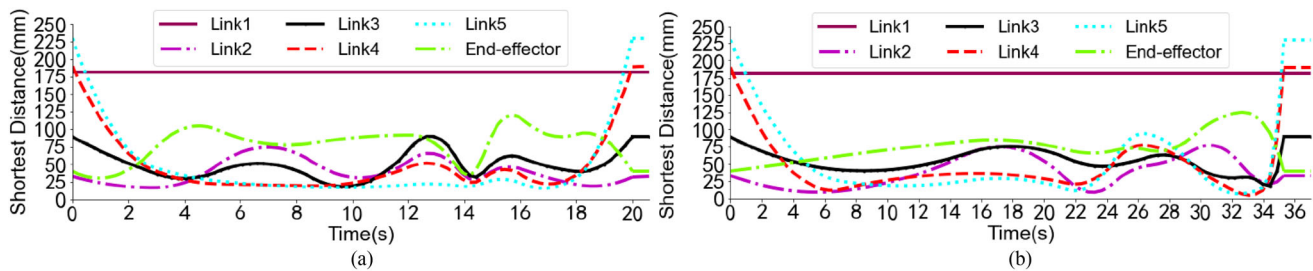


Fig. 17 A plot of the distance between the respective links and the nearest object that it can collide with in the final path. **a** vs.-RRT*FN. **b** RRT_Connect

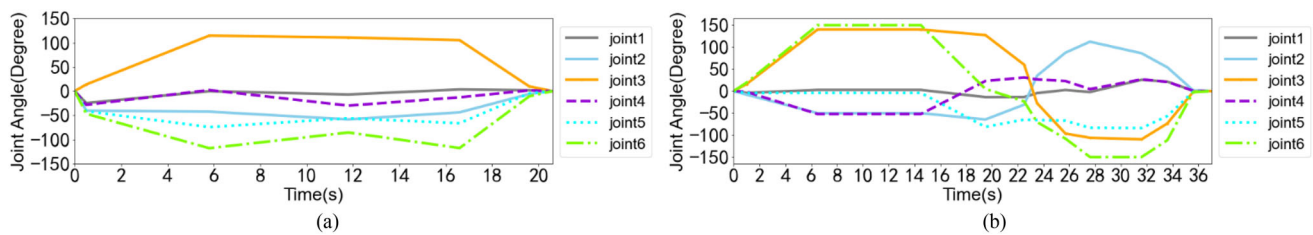


Fig. 18 Variation in the joint angle of task T. **a** vs.-RRT*FN. **b** RRT_Connect

Table 4 Results of completing task T under the control of RRT, RRT*FN, RRT_Connect and vs.-RRT*FN using ROS

Algorithm	Average run time (s)	Average change angle of the links (rad)	Maximum change angle of the links (rad)	Success rate (%)
RRT*	72.7	6.1	7.6	12
RRT*FN	76.3	5.9	8.0	14
RRT_Connect	40.9	5.6	7.3	69
vs.-RRT*FN	17.6	2.7	2.8	100

The smaller average rotation angle of the entire joint planned by the manipulator at one time indicates that the algorithm has a stronger ability to optimize the path and saves energy. A small difference between the maximum turning angle and the average turning angle indicates that the robotic arm is more stable.

Conclusion

The RRT algorithm samples a large number of redundant nodes in the search process. Because of the sampling in the entire space and use of a random sampling strategy, the random points are in a disorderly distribution state. Hence, the planning efficiency is low. The standard RRT-Connect algorithm introduced a dual-tree strategy, so the number of nodes sampled is smaller, and the size of the random tree generated is also smaller. However, the double tree grows randomly and does not use environmental information. The path is even more tortuous than that with the standard RRT algorithm, and there is also the problem of excessive exploration of the space behind the random tree. The improved RRT_Connect algorithm combines the artificial potential field method and RRT-Connect. Because the improved potential field function is applied based on uniform sampling, the sampling point will be guided by the target point and obstacles, which reduces the influence of the random component. Compared to RRT and RRT-Connect, the sampling points created by improved RRT_Connect are more concentrated and clustered near the target point. In addition, because RRT* uses an iterative path optimization strategy, the path is flatter than that generated by the former algorithms. Due to the adoption of the more effective sampling, greedy expansion and double expansion

strategies, the improved algorithm is significantly better than the standard RRT and RRT-Connect in terms of sampling nodes, generated search trees, and planned paths. Although the sampling points of improved RRT_Connect are more concentrated, and the path segment before the end point is flatter, due to the influence of the target point and obstacles nearby, the random tree growth oscillates obviously, and the number of sampling points is also larger than that with the proposed algorithm. Intensive sampling near the target point further leads to many branches near the end point, which will increase the time cost of collision detection. But, the sampling nodes of our method are mainly distributed in the goal guide area and have the lowest density. Besides, owing to the adoption of improvements for the introduction of the boundary extension method, it will not fall into the local minimum and can easily and smoothly pass through the narrow channel, and there is almost adaptability to various scenarios.

In summary, this paper proposes an improved RRT*FN manipulator path planning algorithm that can adapt to various scenarios to determine optimal paths in real time. The algorithm is improved on the basis of the RRT*FN framework and includes heuristic sampling, dichotomy greedy expansion, two additional expansions, local environment sampling boundary expansion and triangle inequality path optimization. The function and role of each module are discussed in detail in the corresponding sections. The fusion of these modules enhances the adaptability of the algorithm to various scenarios. The algorithm was tested in two representative environments, namely, a two-dimensional space and a three-dimensional space, and was also applied to a six-DOF manipulator in an ROS environment. The results verify the ability of the algorithm to realize obstacle avoidance path planning in various scenarios while satisfying the obstacle

avoidance requirements. Compared with algorithms such as RRT*FN, the average search time of the proposed algorithm is an order of magnitude lower, and the path length is shorter in all environments, particularly in the narrow-channel environment. The success rate of the proposed algorithm is quite high.

The planned next step is to investigate the impact of preset hyperparameter values, such as the reserved safety distance, on specific environments and to develop an efficient method for eliminating the influence of these hyperparameters on the determination of the optimal path. The influence of the important path planning parameters can be studied, particularly how the expansion step size can be dynamically adjusted according to the surrounding environment to accelerate the convergence in different environments. Moreover, the proposed manipulator path planning algorithm is based on a static obstacle environment, so the boundary points need to be expanded using the *Boundary Extend* boundary extension method only once. In future work, the manipulator obstacle avoidance algorithm should be studied in dynamic obstacle environments to ensure that the algorithm can run freely in both dynamic and static environments. Furthermore, the use of the previously mentioned short-distance local reinforcement learning method to solve the singular value problem of inverse kinematics must still be investigated experimentally.

Acknowledgements This work is partially supported by the National Natural Science Foundation of China (Project No. 52065010 and No. 52165063), Guizhou Provincial Key Technology R&D Program (Project No. [2023] G094 and No. [2023] G125), Guizhou Provincial Basic Research Program (Natural Science) (No. [2022] G140), Guizhou Provincial Major Scientific and Technological Program (No. [2022] K024), and Laboratory Open Project of Guizhou University (SYSKF2023-089). In addition, thanks for the computing support of the State Key Laboratory of Public Big Data, Guizhou University.

Author contributions JQ contributed to conceptualization, methodology, investigation, formal analysis, writing original draft and visualization. QY helped perform conceptualization, methodology, writing review and editing. CW helped perform investigation, formal analysis. XD helped perform investigation, formal analysis. FD helped perform the analysis with constructive discussions for paper revisions. AR helped perform material preparation with constructive discussions for paper revisions.

Availability of data and materials All data generated or analyzed during this study are included in this published article [and its supplementary information files].

Code availability All data, models, and code generated or used during the study appear in the submitted article.

Declarations

Conflict of interest We declare that we do not have any competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethical approval Not applicable.

Consent to participate Jianyou Qi, Qingni Yuan, Chen Wang, Xiaoying Du, Feilong Du, Ao Ren consent to participate.

Consent for publication Jianyou Qi, Qingni Yuan, Chen Wang, Xiaoying Du, Feilong Du, Ao Ren consent for publication.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Yu YH, Zhang YT (2022) Collision avoidance and path planning for industrial manipulator using slice-based heuristic fast marching tree. *Robot Comput-Integr Manuf* 75:102289. <https://doi.org/10.1016/j.rcim.2021.102289>
2. Khan AH, Li S, Luo X (2020) Obstacle avoidance and tracking control of redundant robotic manipulator: an RNN-based meta-heuristic approach. *IEEE Trans Ind Inf* 16(07):4670–4680. <https://doi.org/10.1109/TII.2019.2941916>
3. Xu ZH, Zhou XF, Wu HM, Li XX, Li S (2022) Motion planning of manipulators for simultaneous obstacle avoidance and target tracking: an RNN approach with guaranteed performance. *IEEE Trans Ind Electron* 69(04):3887–3897. <https://doi.org/10.1109/TIE.2021.3073305>
4. Zhou CH, Tao HF, Chen YY, Stojanovic V, Paszke W (2022) Robust point-to-point iterative learning control for constrained systems: a minimum energy approach. *Int J Robust Nonlinear Control* 32(18):10139–10161. <https://doi.org/10.1002/rnc.6354>
5. Zhuang ZH, Tao HF, Chen YY, Stojanovic V, Paszke W (2022) Iterative learning control for repetitive tasks with randomly varying trial lengths using successive projection. *Int J Adapt Control Signal Process* 36(5):1196–1215. <https://doi.org/10.1002/acs.3396>
6. Song XN, Sun P, Song S, Stojanovic V (2022) Event-driven NN adaptive fixed-time control for nonlinear systems with guaranteed performance. *J Franklin Inst-Eng Appl Math* 359(9):4138–4159. <https://doi.org/10.1016/j.jfranklin.2022.04.003>
7. Zhang Z, Li XY, Dong HZ, Zhou L, Gao L (2022) Constrained sampling method based RRT algorithm for manipulator motion planning. *Comput Integr Manuf Syst* 28(06):1616–1626. <https://doi.org/10.13196/j.cims.2022.06.002>
8. Li Y, Xu D (2020) Cooperative path planning of dual-arm robot based on attractive force self-adaptive step size RRT. *Robot* 42(05):606–616. <https://doi.org/10.13973/j.cnki.robot.190592>
9. Wang HZ, Gao M, Wang JH, Fang LJ, Li HS (2022) Multi-scene fast motion planning of manipulator based on improved RRT*-connect algorithm. *Trans Chin Soc Agric Mach* 53(04):432–440
10. Zhang XB, Liu JG, Li YM (2022) An obstacle avoidance algorithm for space hyper-redundant manipulators using combination of RRT and shape control method. *Robotica* 40(4):1036–1069. <https://doi.org/10.1017/S0263574721000928>

11. Karaman S, Frazzoli E (2010) Optimal kinodynamic motion planning using incremental sampling-based methods. In: 49th IEEE conference on decision and control (CDC), pp7681–7687. <https://doi.org/10.1109/CDC.2010.5717430>.
12. Zhou Y, Zhang ED, Guo HL, Zhang ED, Guo HL, Fang YH, Li H (2021) Lifting path planning of mobile cranes based on an improved RRT algorithm. *Adv Eng Inform* 50:101376. <https://doi.org/10.1016/j.aei.2021.101376>
13. Adiyatov O, Varol HA (2013) Rapidly-exploring random tree based memory efficient motion planning. In: 2013 IEEE international conference on mechatronics and automation, pp 354–359. <https://doi.org/10.1109/ICMA.2013.6617944>.
14. Tian LJ, Zhang ZZ, Zheng CG, Tian Y, Zhao YC, Wang ZY, Qin YH (2021) An improved rapidly-exploring random trees algorithm combining parent point priority determination strategy and real-time optimization strategy for path planning. *Sensors* 21(20):6907. <https://doi.org/10.3390/s21206907>
15. Shen HH, Xie WF, Tang JY, Zhou T (2023) Adaptive manipulability-based path planning strategy for industrial robot manipulators. *IEEE/ASME Trans Mechatron*. <https://doi.org/10.1109/TMECH.2022.3231467>
16. Liu YL, Zuo GY (2020) Improved RRT Path Planning Algorithm for Humanoid Robotic Arm. In: 2020 Chinese Control and Decision Conference (CCDC), pp397–402. <https://doi.org/10.1109/CCDC49329.2020.9164659>.
17. Yi JH, Yuan QN, Sun RT, Bai H (2022) Path planning of a manipulator based on an improved P_RRT* algorithm. *Complex Intell Syst* 8(3):2227–2245. <https://doi.org/10.1007/s40747-021-00628-y>
18. Li YZ, Wang ST, Jiang LQ, Meng J, Xie YL (2021) Motion planning of mobile manipulators based on RRT with sparse nodes. *China Mech Eng* 32(12): 1462–1470. <http://www.cmemo.org.cn/EN/Y2021/V32/I12/1462>.
19. Yang W, Wen HY, Zhang ZS (2021) Obstacle avoidance path planning of manipulator based on improved RRT algorithm. In: 2021 International Conference on Computer, Control and Robotics (ICCCR), pp104–109. <https://doi.org/10.1109/ICCCR49711.2021.9349398>.
20. Fang LJ, Wu ZH, Wang HZ (2021) Multi-scene motion planning of manipulators based on improved RRT* FN algorithm. *China Mech Eng* 32(21):2590–2597
21. Liu C, Feng QC, Tang ZL, Wang XY, Geng JP, Xu LJ (2022) Motion planning of the citrus-picking manipulator based on the TO-RRT algorithm. *Agriculture* 12(5):581. <https://doi.org/10.3390/agriculture12050581>
22. Jiang LH, Liu SY, Cui YM, Jiang HX (2022) Path planning for robotic manipulator in complex multi-obstacle environment based on improved_RRT. *IEEE/ASME Trans Mechatron* 27(6):4774–4785. <https://doi.org/10.1109/TMECH.2022.3165845>
23. Kuffner JJ, LaValle SM (2000) RRT-connect: an efficient approach to single-query path planning. *Proceedings 2000 ICRA. Millennium Conference. In: IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065) 2: 995–1001*. <https://doi.org/10.1109/ROBOT.2000.844730>.
24. Wu ZP, Meng ZJ, Zhao WL, Wu Z (2021) Fast-RRT: a RRT-based optimal path finding method. *Appl Sci* 11(24):11777. <https://doi.org/10.3390/app112411777>
25. Li BH, Chen BD (2021) An adaptive rapidly-exploring random tree. *IEEE/CAA J Autom Sin* 9(2):283–294. <https://doi.org/10.1109/JAS.2021.1004252>
26. Zhang L, Huang SZ, Du ZC, Ouyang GY, Chen HP (2022) Motion-planning algorithm for a hyper-redundant manipulator in narrow spaces. *Comput Mater Continua* 72(3):4817–4832. <https://doi.org/10.32604/cmc.2022.026845>
27. Wang JP, Liu SR, Zhang BT, Yu CB (2021) Manipulation planning with soft constraints by randomized exploration of the composite configuration space. *Int J Control Autom Syst* 19(3):1340–1351. <https://doi.org/10.1007/s12555-019-0727-8>
28. Wang DX, Zhang X, Zhang YR, Xiao J (2012) Configuration-based optimization for six degree-of-freedom haptic rendering for fine manipulation. *IEEE Trans Haptics* 6(2):167–180. <https://doi.org/10.1109/TOH.2012.63>
29. Li XJ, Liu HS, Dong MH (2021) A general framework of motion planning for redundant robot manipulator based on deep reinforcement learning. *IEEE Trans Ind Inf* 18(8):5253–5263. <https://doi.org/10.1109/TII.2021.3125447>
30. Adiyatov O, Varol HA (2017) A novel RRT*-based algorithm for motion planning in dynamic environments. In: 2017 IEEE International Conference on Mechatronics and Automation (ICMA), pp 1416–1421. <https://doi.org/10.1109/ICMA.2017.8016024>
31. Kang G, Kim YB, Lee YH, Oh HS, You WS, Choi HR (2019) Sampling-based motion planning of manipulator with goal-oriented sampling. *Intel Serv Robot* 12(3):265–273. <https://doi.org/10.1007/s11370-019-00281-y>
32. Noreen I, Khan A, Ryu H, Doh NL, Habib Z (2018) Optimal path planning in cluttered environment using RRT*-AB. *Intel Serv Robot* 11(1):41–52. <https://doi.org/10.1007/s11370-017-0236-7>
33. liang Y, Mu HY, Chen DAS, Wei XD, Wang M (2020) PR-RRT*: motion planning of 6-DOF robotic arm based on improved RRT algorithm. In: 2020 10th Institute of Electrical and Electronics Engineers International Conference on Cyber Technology in Automation, Control, and Intelligent Systems (CYBER), pp 417–422. <https://doi.org/10.1109/CYBER50695.2020.9279164>
34. Xiang QQ, Chen M, Hong MX, Zhu HY, Yuan WD (2020) Three-dimensional path planning of manipulator based on improved RRT algorithm. In: 2020 Chinese Control and Decision Conference (CCDC), pp 3689–3695. <https://doi.org/10.1109/CCDC49329.2020.9164149>
35. Pan J, Chitta S, Manocha D (2012) FCL: a general purpose library for collision and proximity queries. In: 2012 IEEE International Conference on Robotics and Automation, pp 3859–3866. <https://doi.org/10.1109/ICRA.2012.6225337>
36. Ketchel J, Larochelle P (2006) Collision detection of cylindrical rigid bodies for motion planning. In: *Proceedings 2006 IEEE International Conference on Robotics and Automation (ICRA)*, pp 1530–1535. <https://doi.org/10.1109/ROBOT.2006.1641925>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.